

Arithmétiques pour le traitement numérique du signal

SN424-SN425: RADIO LOGICIELLE

Sommaire

2

- ▶ Les types de variables en C
- ▶ Les nombres entiers
- ▶ Virgule flottante (IEEE 754)
- ▶ Virgule fixe
- ▶ *Logarithmic Number System*
- ▶ Comparaison des arithmétiques à virgule
- ▶ Accélérateurs matériels

Les types de variables en C

Les types de variables en C

Les types de variables en C (1/2)

- ▶ Les types standards en C pour l'arithmétique:
 - ▶ **short**: Entier signé sur 16 bits
 - ▶ **unsigned short**: Entier non signé sur 16 bits
 - ▶ **int**: Entier signé sur 32 bits (ou 16 bits)
 - ▶ **unsigned int**: Entier non signé sur 32 bits (ou 16 bits)
 - ▶ **long**: Entier signé sur 64 bits
 - ▶ **unsigned long**: Entier non signé sur 64 bits
 - ▶ **float**: Nombre flottant simple précision sur 32 bits (IEEE 754)
 - ▶ **double**: Nombre flottant double précision sur 64 bits (IEEE 754)

Les types de variables en C

Les types de variables en C (2/2)

- ▶ Librairie `<stdint.h>`:
 - ▶ `int8_t`: Entier signé sur 8 bits
 - ▶ `int16_t`: Entier signé sur 16 bits
 - ▶ `int32_t`: Entier signé sur 32 bits
 - ▶ `int64_t`: Entier signé sur 64 bits (selon implémentation)
 - ▶ `uint8_t`: Entier non signé sur 8 bits
 - ▶ `uint16_t`: Entier non signée sur 16 bits
 - ▶ `uint32_t`: Entier non signé sur 32 bits
 - ▶ `uint64_t`: Entier non signé sur 64 bits (selon implémentation)

Les nombres entiers

Les nombres entiers

Les types de nombres entiers

- ▶ Les nombres entiers sont les nombres les plus basiques à représenter sur un ordinateur.
- ▶ Ils peuvent être:
 - ▶ Non signé: Entier positif
 - ▶ Signé: Entier positif ou négatif

Les nombres entiers

Les nombres entiers non signés

- ▶ Un nombre entier non signé sur N bits permet de représenter un nombre entre 0 et $2^N - 1$.
- ▶ On peut représenter un nombre entier x non signé ainsi:

$$x = \sum_{n=0}^{N-1} x_n 2^n$$

- ▶ Exemple: Un **unsigned short** (ou **uint16_t**) permet de représenter nombre entier entre 0 et 65535.



Nombre entier non signé (16 bits)

Les nombres entiers

Les nombres entiers signés

- ▶ Un nombre entier signé (en complément à 2) sur N bits permet de représenter un nombre entre -2^{N-1} et $2^{N-1} - 1$.
 - ▶ Le bit de poids fort $x^{(N-1)}$ indique si le nombre est positif ($x_{N-1} = 0$) ou négatif ($x_{N-1} = 1$).

- ▶ On peut représenter un nombre entier x signé ainsi:

$$x = -x_{N-1}2^{N-1} + \sum_{n=0}^{N-2} x_n 2^n$$

- ▶ Exemple: Un **short** (ou **int16_t**) permet de représenter nombre entier entre -32768 et 32767 .
- ▶ Pour obtenir l'opposé d'un nombre x , on utilise le complément à deux:

$$\begin{aligned}\bar{x} + x &= -1 \\ \Leftrightarrow \bar{x} + 1 &= -x\end{aligned}$$

Arithmétique pour le TNS

Les autres types de représentation signée

Nombre	Signe et valeur absolue	Binaire décentré	Complément à 1	Complément à 2
+3	011	111	011	011
+2	010	110	010	011
+1	001	101	001	001
+0	000	100	000	000
-0	100	-	111	-
-1	101	011	110	111
-2	110	010	101	110
-3	111	001	100	101
(-4)		(000)		(100)

Virgule flottante (IEEE 754)

Virgule flottante (IEEE 754)

Définition

12

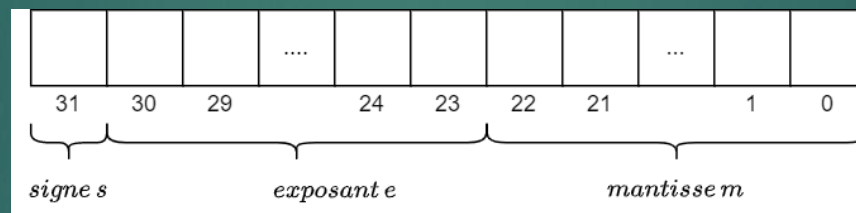
- ▶ Définition: La norme IEEE 754 définit la représentation des nombres à virgule flottante simple précision (**float**) ou double précision (**double**).
- ▶ Un nombre à virgule flottante est composé de trois parties:
 - ▶ Le bit de signe s : Le signe du nombre
 - ▶ L'exposant e : L'ordre de grandeur du nombre
 - ▶ La mantisse m : La précision du nombre

Virgule flottante (IEEE 754)

Représentation

- Représentation d'un nombre flottant simple précision (**float**):

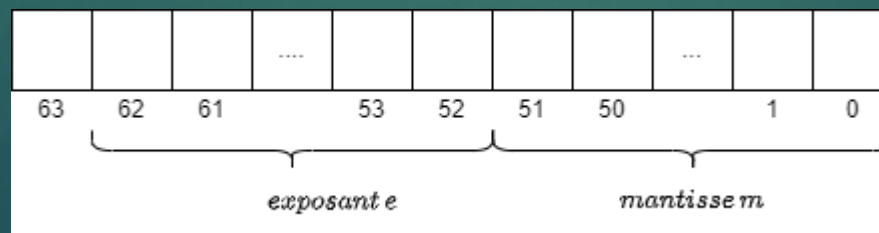
$$x = (-1)^s 2^{e-127} m$$



Représentation simple précision

- Représentation d'un nombre flottant double précision (**double**):

$$x = (-1)^s 2^{e-1023} m$$



Représentation double précision

- $$\begin{aligned} x &= (-1)^0 \times \left(1 + \frac{0}{2^{23}}\right) \times 2^{127-127} \\ &= 1 \times 1 \times 2^0 \end{aligned}$$

Convertisseur de nombre flottant IEEE 754 (en ligne)

Virgule fixe

Virgule fixe

Définition

- ▶ Définition: La représentation à virgule fixe permet de représenter un nombre à virgule (v) à l'aide d'une représentation entière (e).
- ▶ Un nombre à virgule fixe est composé de deux parties:
 - ▶ La partie entière (N_e bits)
 - ▶ La partie fractionnaire (N_f bits)
- ▶ Remarque: On note souvent les nombres à virgules fixes $QN_e.N_f$ (ex: Q1.15) [1]

Virgule fixe

Représentation

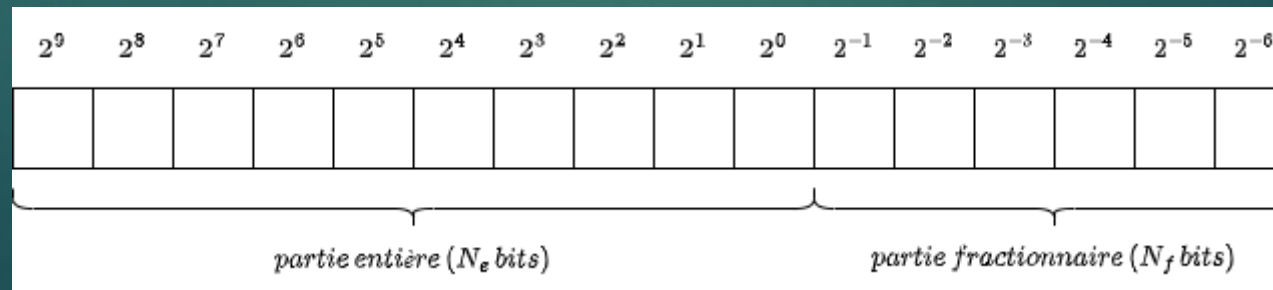
- Représentation d'un nombre à virgule fixe:

$$x = e \cdot 2^{-N_f}$$

Avec e : un nombre entier (non signé ou signé) et 2^{-N_f} : le facteur d'échelle



Nombre entier non signé (16 bits)



Nombre à virgule fixe non signé (16 bits avec 6 bits de partie fractionnaire)

Virgule fixe

Exemples (Q1.15)

- ▶ $x = 0.75$ (0x6000, $N_e=1$ bit, $N_f=15$, $e = 24576$):

$$x = 24576 \cdot 2^{-15}$$
- ▶ $x = -0.5$ (0xc000, $N_e=1$ bit, $N_f=15$, $e = -16384$):

$$x = -16384 \cdot 2^{-15}$$

Q-format Converter & Calculator

- Convert between fractional values and their $Qm.n$ fixed-point integer representations, and calculate the representation error.
- Perform arithmetic operations on fixed-point values.

Please report any issues on [GitHub](#).

Converter | dB Converter | Calculator | Information

Q-Format Settings

Choose your conversion options. [Remember these options.](#)

Word size in bits ($m+n$): 16

Fractional bits (n): 15

Signed? ☒

Integer format: Hexadecimal

Maximum fractional value: 0.999969482421875

Minimum fractional value: -1

Resolution: 0.000030517578125

Bit positions and values

Bit Position	15	14	13	2	1	0
Bit Value	-2^0	2^{-1}	2^{-2}	2^{-13}	2^{-14}	2^{-15}

Convert

Type an integer or fractional value in the boxes below. Click or tab away to update.

Integer (hexadecimal): 6000

Fractional value: 0.75

Fixed-point value: 0.75

Representation error: 0

Representation error (dB): -Infinity

Batch Convert

Type integers or fractional values in the boxes below separated by commas, spaces or new lines. Click or tab away to update.

Integers

Fractional values

Virgule fixe

Opérations avec des virgules fixes

- ▶ Supposons trois nombres:
 - ▶ $n_1 = e_1 \cdot 2^{-N_1}$: Nombre à virgule fixe avec N_1 bits de partie fractionnaire
 - ▶ $n_2 = e_2 \cdot 2^{-N_2}$: Nombre à virgule fixe avec N_2 bits de partie fractionnaire
 - ▶ $n_3 = e_3 \cdot 2^{-N_3}$: Nombre à virgule fixe avec N_3 bits de partie fractionnaire
- ▶ Addition en virgule fixe $n_1 + n_2$:
$$e_3 = \lfloor e_1 \cdot 2^{-(N_1-N_3)} + e_2 \cdot 2^{-(N_1-N_3)} \rfloor$$
- ▶ Soustraction en virgule fixe $n_1 - n_2$:
$$e_3 = \lfloor e_1 \cdot 2^{-(N_1-N_3)} - e_2 \cdot 2^{-(N_1-N_3)} \rfloor$$
- ▶ Multiplication en virgule fixe $n_1 \times n_2$:
$$e_3 = \lfloor (e_1 \times e_2) \cdot 2^{-(N_1+N_2-N_3)} \rfloor$$
- ▶ Division en virgule fixe n_1/n_2 :
$$e_3 = \lfloor (e_1/e_2) \cdot 2^{-(N_1-N_2-N_3)} \rfloor$$

→ Il y a de nombreuses optimisations possibles pour limiter les troncatures $\lfloor \cdot \rfloor$

Virgule fixe

Dimensionnement d'un RIF

- Pour un RIF, le nombre de bits supplémentaires S est égal à:

$$S = \left\lceil \log_2 \left(\sum_{i=0}^M |b_i| \right) \right\rceil$$

→ Si le signal d'entrée $x[n]$ est représenté à l'aide d'une arithmétique à virgule fixe $QN_e.N_f$, il faudra représenter le signal de sortie $y[n]$ à l'aide d'une arithmétique à virgule fixe $QN_e + S.N_f$ pour éviter les dépassement d'entiers (*overflow*).

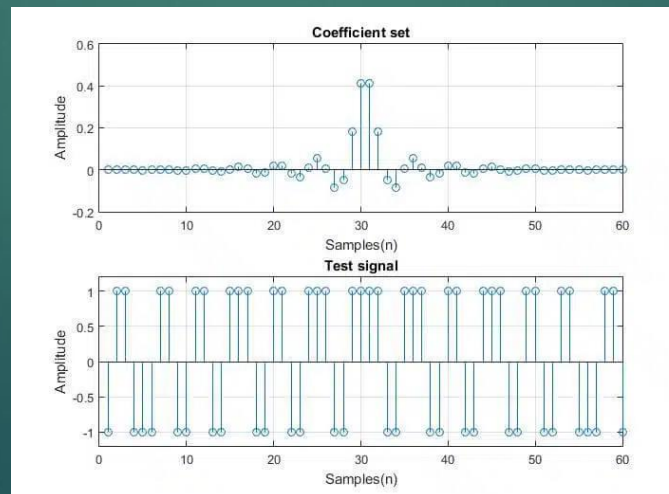


Illustration du signal d'entrée permettant de tester un RIF ([VHDLwhiz](#))

Logarithmic Number System

Logarithmic Number System

Définition

- Définition: Le Logarithmic Number System (LNS) est une arithmétique qui représente un nombre x à l'aide de son signe s_x et du logarithme de sa valeur absolue l_x .

$$x \rightarrow \begin{cases} s_x = \text{signe}(x) \\ l_x = \log_b(|x|) \end{cases}$$

- L'arithmétique LNS est moins répandue que l'arithmétique à virgule flottante ou à virgule fixe.

Arithmétique pour le TNS

Opérations avec des nombres LNS

► Opérations:

- Multiplication: $z = x \cdot y$

$$\begin{cases} s_z = s_x \cdot s_y \\ l_z = l_x + l_y \end{cases}$$

- Division: $z = x/y$

$$\begin{cases} s_z = s_x \cdot s_y \\ l_z = l_x - l_y \end{cases}$$

- Puissance (et racine): $z = x^a$

$$\begin{cases} s_z = s_x^a \\ l_z = a \cdot l_x \end{cases}$$

→ Néanmoins, l'addition et la soustraction sont beaucoup plus complexes à calculer.

Comparaison des arithmétiques à virgule

Comparaison des arithmétiques à virgule

25

Arithmétique	Virgule flottante	Virgule fixe	LNS
Principe	Bit de signe Mantisse Exposant	Partie entière Partie fractionnaire	Signe Logarithme de la valeur absolue
Avantages	Précision variable Dynamique	Calcul entier	Multiplication/Division Puissance/Racine
Inconvénients	Complexité calculatoire	Précision fixe <i>Overflow</i>	Addition/Soustraction

Comparaison des arithmétiques pour représenter des nombres à virgule

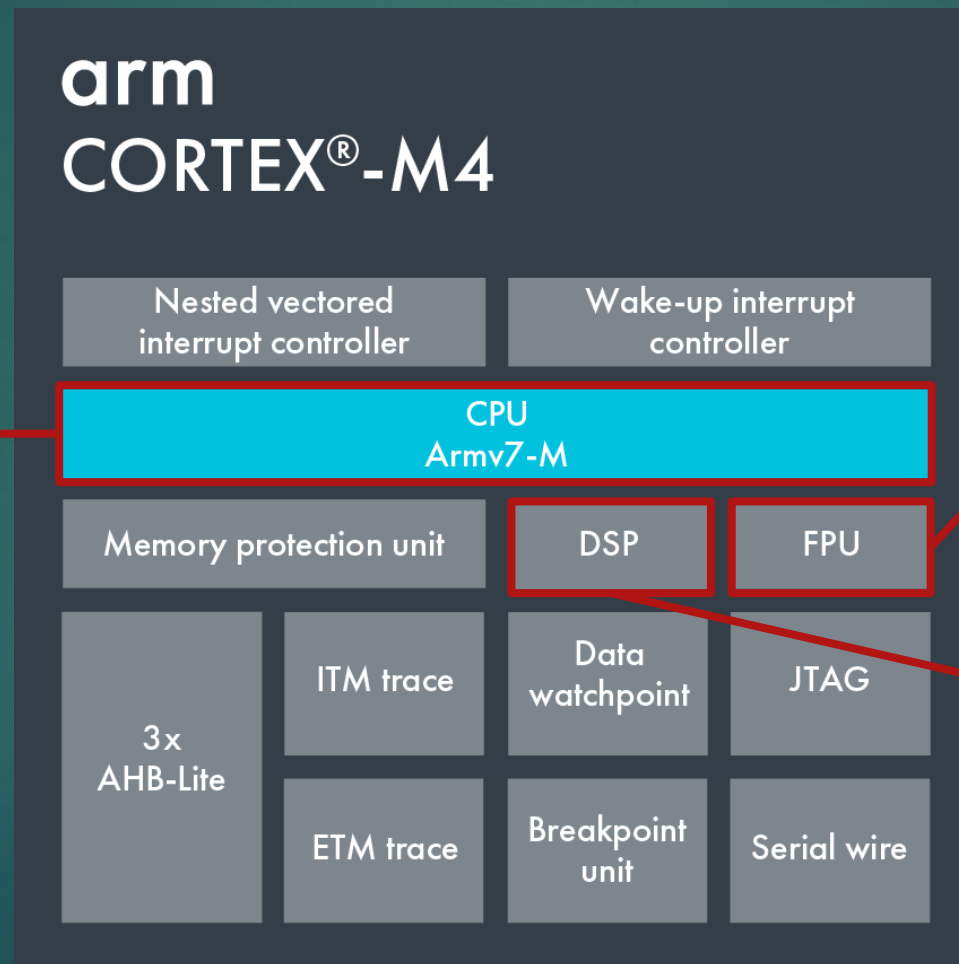
→ Si une unité de calcul en virgule flottante, ou *Floating Point Unit* (FPU) est disponible, l'arithmétique à virgule flottante est à favoriser par rapport à une arithmétique à virgule fixe (ou même LNS).

Accélérateurs matériels

Accélérateurs matériels

Cortex-M4

Le CPU dispose d'une unité de calcul arithmétique et logique (ALU)



Unité de calcul en virgule flottante (FPU) simple précision (IEEE 754)

Jeu d'instructions dédié au traitement du signal (DSP: *Digital Signal Processing*): Multiplieur-ACcumulateur (MAC), *Single Instruction Multiple Data* (SIMD)

Accélérateurs matériels

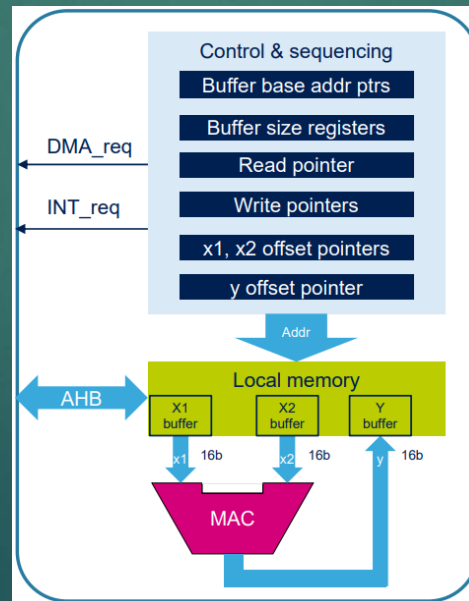
CMSIS-DSP

- ▶ *Common Microcontroller Software Interface Standard* (CMSIS) est un ensemble d'APIs, d'outils et de bibliothèques pour développer du code sur Cortex-M et Cortex-A.
- ▶ Il existe notamment des bibliothèques le déploiement d'algorithmes de traitement du signal (CMSIS-DSP) ou de réseaux de neurones (CMSIS-NN) sur microcontrôleur.
- ▶ CMSIS-DSP propose de nombreuses fonctions pour l'implémentation logicielle de filtres:
 - ▶ **arm_fir_f32**: Implémentation logicielle d'un RIF en virgule flottante simple précision
 - ▶ **arm_fir_fast_q15**: Implémentation logicielle rapide (SIMD) d'un RIF en virgule fixe (Q1.15)
 - ▶ **arm_biquad_cascade_df1_f32**: Implémentation logicielle d'un RII en virgule flottante simple précision à l'aide de sections biquadratiques
 - ▶ **arm_biquad_cascade_df1_fast_q15**: Implémentation logicielle rapide (SIMD) d'un RII en virgule fixe (Q1.15) à l'aide de sections biquadratiques
 - ▶ ...

Accélérateurs matériels

FMAC

- ▶ Certains microcontrôleurs STM32 (STM32G4, STM32H7, ...) dispose d'un accélérateur matériel dédié aux filtrages numériques (RIF ou RII): le *Filter Math ACcelerator* (FMAC).
- ▶ Cet accélérateur est centré autour d'un MAC de 16 bits (Q1.15) et de *buffers* circulaires.



Composition d'un FMAC [4]

Accélérateurs matériels

Comparaison CMSIS-DSP et FMAC

- Configuration du benchmark:
 - Plateforme: STM32G474 à 150 MHz
 - IDE: IAR EWARM 7.2 avec des optimisations haute performances
 - Cas d'étude: Un RIF avec $M=51$ et un RII à $M=3$ et $N=2$
- Les fonctions de CMSIS-DSP sont en moyennes 15% plus rapides que les calculs réalisés par le FMAC, dû aux capacités SIMD (*Single Instruction Multiple Data*) du Cortex-M4. Cependant, l'utilisation conjointe du FMAC avec la DMA (*Direct Memory Access*) permet de décharger le CPU, car les calculs sont réalisés en arrière-plan.

Filter	CMSIS DSP (software) function	CMSIS DSP execution time (clock cycle/sample)	FMAC execution time (clock cycle/sample)
Single stage bi-quad/2p2z (IIR with $N=3$, $M=2$)	arm_biquad_cascade_df1_fast_q15()	12	14
51-tap FIR ($N=51$)	arm_fir_fast_q15()	92	104

Merci de votre écoute

Bibliographie

32

- ▶ [1] M. Puech – Traitement du signal embarqué
- ▶ [2] Arm® Cortex®-M4 in a nutshell (st.com)
- ▶ [3] CMSIS DSP Software Library (Keil.com)
- ▶ [4] STM32G4 – FMAC (st.com)